

The

C

For You

The Ultimate C Language Guide

printf(“Basic to Advanced++”);

Chapter 1

Basic of C Language

(Detailed Notes, MCQ, SAQ, Sample Programs)

{Unlock The Depth Of C}

✦ **Pritam Pandit** ✦

contact.pritam1926@gmail.com | Ph: 7719332510

Contents

Chapter 1: Basics of C Language –

1.1 Introduction to C

- History of C Language (Developed by Dennis Ritchie in 1972)
- Importance & features of C:
- Applications of C:

1.2 Structure of a C Program

1.3 Writing and Compiling a Program

1.4 C Tokens

- Keywords (e.g., int, return, if)
- Identifiers (e.g., variable names)
- Constants (e.g., 10, 3.14)
- Operators (e.g., +, -, *)
- Special symbols ({ }, ;, ,)
- Strings ("Hello")

1.5 Identifiers

1.6 Variables

1.7 Keywords

1.8 Constants

1.9 Data Types

- Basic Data Types:
 - int, float, char, double
- Type modifiers:
 - short, long, signed, unsigned
- Size of data types (platform dependent)

1.10 Type Conversion

- Implicit type conversion (automatic)
- Explicit type conversion (typecasting)

1.11 Escape Sequence

1.12 Comments in C

- Single-line comment
- Multi-line comment

1.13 Console Input and Output Functions

- printf() – for output
- scanf() – for input

MCQs

SAQs

Sample Programs

1.1 Introduction to C

C is a general-purpose, **procedural programming language**, developed by **Dennis Ritchie** in **1972** at **Bell Labs**.

C is also known as “**mother of all modern languages**” (like C#, C++, Java, Python).

Features of C

- ❖ Simple & Efficient: - Minimal keywords, close to hardware
- ❖ Fast Execution: - Compiled language → High performance
- ❖ Portable: - Same code can run on different machines
- ❖ Modular: - Supports functions and code separation
- ❖ Rich Library: - Has built-in functions for I/O, math, string etc.
- ❖ Middle-Level: - Has low-level (pointer, memory) + high-level (functions, loops) features

What Can You Do With C

- ❖ Build operating systems (e.g., UNIX)
- ❖ System-level programming
- ❖ Embedded systems & microcontrollers
- ❖ Compiler and interpreter development
- ❖ Game engines and real-time systems

1.2 Structure of a C Program

```
#include<stdio.h>
int main() {
    printf("Hello Pritam");
    return 0;
}
```

Output:

```
Hello Pritam
```

Breakdown:

- #include <stdio.h> → Standard I/O header file
- int main() → main() function, return int

- `printf()` → Output text to screen
- `return 0` → Indicates successful completion

1.3 C Program Execution Steps

1. **Write** the code → .c file
2. **Compile** it using a compiler (e.g., GCC)
3. **Link** libraries → Creates .exe
4. **Run** the executable → Output on screen

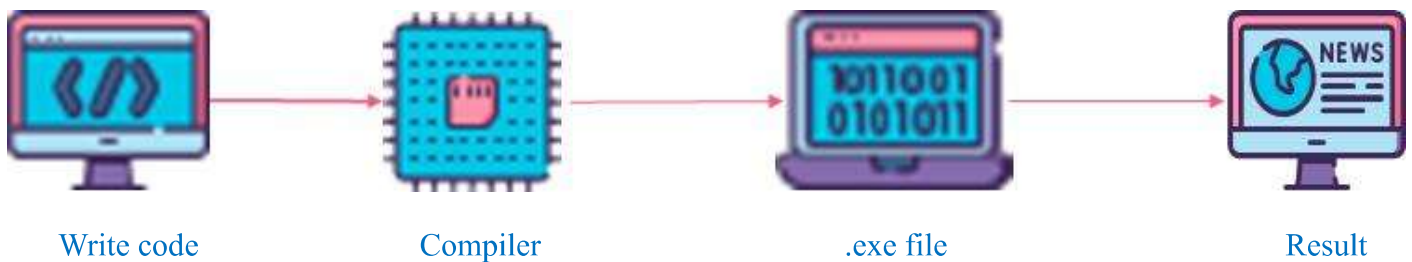


Fig 1.1

1.4 C Tokens

The smallest individual units are known as tokens. **C has six types of tokens.**

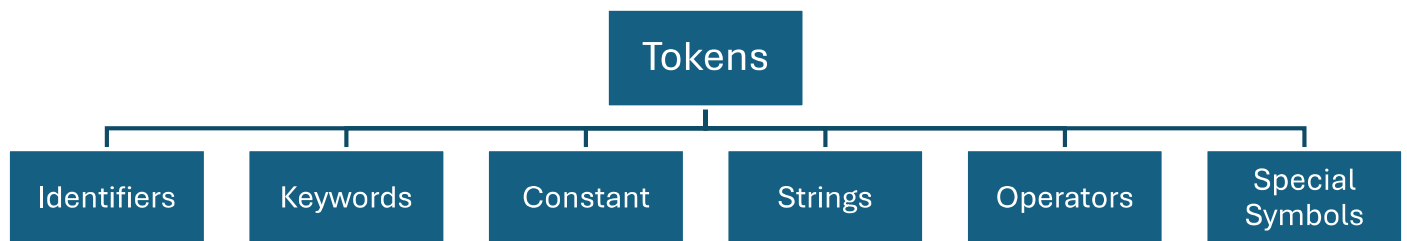


Fig 1.2

1.5 Identifiers

Identifiers refer to the names of **variables, constants, functions and arrays**. These are user-defined names called Identifiers.

Rules for identifiers

- ❖ Only letters, digits, and underscore (`_`) allowed
- ❖ Must start with **letter** or underscore (`_`) not digit
- ❖ **No space**

- ❖ **No C keywords**
- ❖ Case-sensitive (Age $\neq$ age)
- ❖ Up to 31 characters allowed

Example:

Valid

```
Roll _a2 Pritam_Pandit
isFollow() num2 arr[]
```

Invalid

```
int @hi 14z Is follow()
1a[]
```

1.6 Variables

A variable is a name of **memory location**. It is used to store data. Variables are **changeable**, we can change the value of a variable during execution of a program. It can be **reused** many times. It is nothing but an **identifier**.

Rules for variable: same as identifier

Syntax:

```
data_type variable_name = value;
```

Example:

```
int x=100;
float a=12.25;
char m="f";
```

Types of Variables

1. **Local Variable:** - **Declared** inside a function or block, **Scope** only that function/block, **Lifetime** destroyed after block ends.
2. **Global Variable:** - **Declared** outside of all functions (usually top of file), **Scope** accessible in all functions of that file, **Lifetime** stays in memory till the program ends.

```
#include <stdio.h>
int global=5;
int main() {
    int local=10;
    printf("%d %d",global,local);
}
```

Output:

```
5 10
```

Noted

- The declaration of variables must be done before they are used in the program.
- Uninitialized global variable **stores 0**.
- Uninitialized local variable **stores garbage value**.

1.7 Keywords

A keyword is a **reserved word**. All keywords have fixed meanings, that means we cannot change. All keywords must be written in **lowercase**. A list of **32 keywords** in c language is given below:

auto	break	case	char	const	continue	default
do	double	enum	else	extern	float	for
goto	if	int	long	return	register	signed
short	static	sizeof	struct	switch	typedef	union
unsigned	void	volatile	while			

Fig 1.3

1.8 Constant

const is a keyword used to declare a variable as constant, meaning its value can't be changed after it is **initialized**.

Syntax:

```
const data_type variable_name = value;
```

Example:

```
#include <stdio.h>
int main() {
    const int a=5;
    printf("%d", a);
}
```

Output:

```
5
```

Constant using #define

#define directives can also be used to define **symbolic constants** that do not require a **data type**. Works in **compile time**.

Syntax:

```
#define constant_name value
```

Example:

```
#include <stdio.h>
#define PI 3.14
int main() {
    printf("%.2f", PI);
}
```

Output:

```
3.14
```

Noted

- You must initialize constant at the time of declaration.
- #define doesn't know about **data types**.
- #define doesn't take up any **memory** like a const variable does.
- #define is global.

1.9 Data Types

Data type means which data stores in memory. There are 4 types of data types in C.

Types	Data types
Primary DT	int, char, float, double
Derived DT	array, pointer, structure, union
Enumeration DT	enum
Void DT	void

Basic Data Types

These are the most common and fundamental data types.

- **int**: Used to store whole numbers (integers) without any decimal points.
 - **Size**: Typically, 2 or 4 bytes, depending on the system architecture.
 - **Example**: `int age = 25;`
- **char**: Used to store a single character. Characters are enclosed in **single quotes**.
 - **Size**: 1 byte.
 - **Example**: `char initial = 'A';`
- **float**: Used to store numbers with a decimal point (single-precision floating-point numbers). Decimal point number by **default is double**, we add **f/F** end of the number to set it float.
 - **Size**: Typically, 4 bytes.
 - **Example**: `float price = 19.99f;`
- **double**: Used to store numbers with a decimal point but with higher precision than float (double-precision floating-point numbers).
 - **Size**: Typically, 8 bytes.
 - **Example**: `double pi = 3.1415926535;`

Derived Data Types

These are built from or derived from the basic data types.

- **Arrays**: A collection of similar data type elements stored in contiguous memory locations.
- **Pointers**: A variable that stores the memory address of another variable.
- **Structures**: A collection of different data types under a single name.
- **Unions**: Like structures, but all members share the same memory location.

Enumeration Data Type

- **enum**: Used to define a set of named integer constants. This makes code more readable and easier to maintain.
 - **Example**: `enum Weekday { Sunday, Monday, Tuesday };`

Void Data Type

Void means the absence of a value or unspecified type of variable. It is mainly used in **function return type** or in **pointer**.

Type Modifiers

Type modifiers **change the size or range** of a data type. **There are 4 Types of Modifiers:**

1. short
2. long
3. signed
4. unsigned

unsigned → Only positive numbers

signed → Both positive and negative (default)

Size and Ranges of Data Types with Qualifiers

Data Type	Range	Bytes	Format
signed char	-128 to 127	1	%c
unsigned char	0 to 255	1	%c
signed short int	-32768 to +32767	2	%hd
unsigned short int	0 to 65535	2	%hu
signed int	-2147483648 to +2147483647	4	%d
unsigned int	0 to 4294967295	4	%u
signed long int	-2147483648 to +2147483647	4	%ld
unsigned long int	0 to 4294967295	4	%lu
signed long long int	-9223372036854775808 to +9223372036854775807	8	%lld
unsigned long long int	0 to 18446744073709551615	8	%llu
float	-3.4e38 to +3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to +1.7e4932	10	%Lf

💡 Range calculate formula

- For signed: $(-2^n \text{ to } 2^n - 1)$, where n = number of bits.
- For unsigned: $(0 \text{ to } 2^n - 1)$, where n = number of bits.

Octal:

It consists of digits from 0 to 7. It should **prefix with 0**. It allows sign (+,-). No special character is allowed.

Used %o (exp: 015, +014, -013)

Hexa-decimal:

It consists of digits from 0-9, a-f (capital letters & small letters). it should **prefix with 0X or 0x**. It allows sign (+,-). No special character is allowed. **Used %x, %X** (exp: 0x13, 0x4f)

Noted

- Point number by default **double**.
- Type modifiers work only **int and char** Data Type.
- **sizeof()** operator finds the size of any data type on your system.

1.10 Type Conversion (Implicit & Explicit)

Type conversion (also called **typecasting**) means changing the **data type of value or variable**.

❖ **Implicit Type Conversion:** Happens automatically by the **compiler**.

```
#include <stdio.h>
int main() {
    int a = 10;
    float b = 5.5;
    float result = a + b; // int is converted to float
    printf("Result = %.2f\n", result); // Result = 15.50
}
```

❖ **Explicit (Type Casting):** Done manually by the **programmer**. A type is forced into another using casting.

Syntax:

```
(type) expression;
```

```
#include <stdio.h>
int main() {
    float num = 9.99;
    int x = (int)num; // explicit conversion
    printf("x = %d\n", x); //x = 9
}
```

Noted

- int + float → float
- float + double → double
- Promotion order: char < int < float < double.

1.11 Escape Character

It is used to escape the character's real meaning. Used **backslash (\)**

1. \n newline
2. \r carriage return
3. \t tab
4. \v vertical tab
5. \b backspace
6. \f form feed (page feed)
7. \a alert (beep)
8. \' single quote(')
9. \" double quote(“)
10. \? Question mark (?)
11. \\ backslash (\)

1.12 Comments in C

Comments are used to clarify either the purpose of the program or purpose of some statement in the program. **It is ignored by compiler.**

```
//single line comment

/* multiline
Comment*/

“It is also a comment”;
```

1.13 Console I/O functions in C

I/O means **Input/Output**, In C when we perform I/O operation, need I/O function to complete the task. There are 2 types of I/O in C: (a) **Console I/O**, (b) **File I/O**

Here we discuss **console I/O**.

Formatted I/O Functions

Formatted I/O functions operate on various types of data.

(a) printf()

Output data or result of an operation can be displayed from the computer to a **standard output device** using the library function **printf()**. This function is used to print any combination of data. It returns the **number of characters** that it displayed on console window. The return type is **int**.

```
printf(" hello ");      // hello
printf("a");           // a
printf("%d", a);       // value of a
printf("%d");          // no display
printf("hello Pritam"); //return value 12
printf("%d", 125);     //return value 3 (125 is 3 characters)
```

(b) scanf()

Input data can be entered into the computer using the **standard input** C library function called **scanf()**. This function is used to enter any combination of input. Doesn't allow **space** between text. It returns the **number of successful inputs** taken from user. Its return type is **int**.

```
scanf("%d",&a);        // scan int value
scanf("%f",&a);        // scan float value
scanf("%c",&a);        // scan char value
scanf("%d %d",&a,&b);   //input (5, A) return value 1 (A is not int, unsuccessful)
```

Unformatted I/O Functions

These functions deal with single char or string only.

(a) getch()

Reads a single character from the keyboard by the user but **doesn't display that character** on the console screen and **immediately returns** without pressing enter key. Declared in **conio.h** header file.

```
#include <stdio.h>
#include <conio.h>
int main() {
    printf("Enter a char: ");
    getch();
}
```

(b) getche()

Same as getch() but difference is that **display that character** on console screen. Declared in **conio.h** header file.

```
#include <stdio.h>
#include <conio.h>
int main() {
    printf("Enter a char: ");
    getche();
}
```

(c) getchar()

Read only one character at a time until and unless the enter key is pressed. Declared in **stdio.h** header file.

```
#include <stdio.h>
int main() {
    printf("Enter a char: ");
    getchar();
}
```

(d) gets()

Reads a string from user and stores in a character array. Allow us to write **space-separated** text. Declared in **stdio.h** header file.

(e) puts()

Display a string which is already stored in a character array. Declared in **stdio.h** header file.

```
#include <stdio.h>
#include <conio.h>
int main() {
    char name[50];
    printf("Enter a name: ");
    gets(name);
    puts(name);
}
```

(f) putchar()

Display a single character on a console screen. Declared in **conio.h** header file.

```
#include <stdio.h>
#include <conio.h>
int main() {
    printf("Enter a char: ");
    char ch=getch();
    putchar(ch);
}
```

Noted

- I/O operations are carried out through **function calls**.
- In scanf() each variable name must be preceded by an ampersand (&).
- getch() is also used for hold the screen.
- gets() is **deprecated** due to buffer overflow issues.

Chapter 1 MCQs

1. Who developed the C programming language ?

- a) James Gosling b) Dennis Ritchie c) Bjarne Stroustrup d) Ken Thompson
-

2. Which of the following is not a feature of C ?

- a) Structured programming b) Machine independent c) Object-oriented d) Fast and efficient
-

3. Which of the following is a valid identifier in C ?

- a) int b) total_marks c) 2value d) float
-

4. What is the size of int on a 32-bit system (generally) ?

- a) 2 bytes b) 4 bytes c) 8 bytes d) Compiler dependent
-

5. Which of the following is a type modifier in C ?

- a) double b) short c) float d) char
-

6. What will be the output of this program ?

```
#include <stdio.h>
int main() {
    int a = 5, b = 2;
    printf("%d", a / b);
}
```

- a) 2.5 b) 2 c) 2.00 d) Error
-

7. What will the following code print ?

```
#include <stdio.h>
int main() {
    printf("Hello\tWorld\n");
}
```

- a) Hello World b) Hello World c) HelloWorld d) Compilation error
-

8. Which escape sequence is used to print a double quote inside a string ?

- a) \n b) \t c) \" d) \\
-

9. Which of the following is NOT a keyword in C ?

- a) auto b) float c) integer d) goto
-

10. What will be the output of this program ?

```
#include <stdio.h>
int main() {
    float x = 5/2;
    printf("%.1f", x);
}
```

- a) 2.0 b) 2.5 c) 2.00 d) Error
-

11. What will be the output ?

```
#include <stdio.h>
int main() {
    float x = (float)5/2;
    printf("%.1f", x);
}
```

- a) 2.0 b) 2.5 c) 5.0 d) Error
-

12. Which function is used to take input from the user ?

- a) printf() b) scanf() c) cin d) input()
-

13. Which of the following is a constant ?

- a) 3.14 b) total c) value1 d) result
-

14. What will be the output of this code ?

```
#include <stdio.h>
int main() {
    int x = 10;
    printf("%d %d", x, x++);
}
```

- a) 10 11 b) 10 10 c) 11 10 d) Undefined behavior
-

15. Which of the following correctly represents a string in C ?

- a) 'Hello' b) "Hello" c) Hello d) {Hello}
-

Answer Box

1. b	2. c	3. b	4. a	5. b
6. b	7. b	8. c	9. c	10. a
11. b	12. b	13. a	14. c	15. b

Chapter 1 SAQs

1. Who developed the C programming language and in which year ?

The C programming language was developed by **Dennis Ritchie** at Bell Laboratories in **1972**. He created it to build the Unix operating system.

2. Mention any four features of the C language ?

- **Portability:** C programs can be run on different machines with minimal changes.
 - **Speed:** It's a low-level language, which gives it a high execution speed compared to many other languages.
 - **Mid-level Language:** C combines the features of both low-level (like assembly) and high-level languages.
 - **Rich Library:** It has a large collection of built-in functions that make programming easier.
-

3. Write the general structure of a C program ?

A C program typically has the following structure:

- **Preprocessing directives:** (e.g., `#include <stdio.h>`) to include header files.
 - **Global variable declarations:** Variables declared outside any function.
 - **main() function:** The entry point of every C program.
 - **Other functions:** User-defined functions.
-

4. What are C tokens, List different types with examples ?

C tokens are the smallest individual units of a C program. The six main types of C tokens are:

- **Keywords:** `int`, `if`, `for`
 - **Identifiers:** `main`, `age`, `studentName`
 - **Constants:** `10`, `3.14`, `'A'`
 - **Strings:** `"Hello, World!"`
 - **Special symbols:** `{}`, `()`, `[]`
 - **Operators:** `+`, `-`, `*`, `=`
-

5. Define identifiers. Give 2 examples of valid identifiers and 2 invalid identifiers ?

An **identifier** is a name given to a variable, function, array, or any other user-defined item. They must start with an alphabet or an underscore, followed by letters, digits, or underscores. They are also **case-sensitive**.

- **Valid:** myVariable, _count1
 - **Invalid:** 1stNumber (starts with a digit), for (is a keyword)
-

6. Differentiate between keywords and identifiers with examples ?

Keywords are reserved words in C with a predefined meaning, and their meaning cannot be changed. Examples: int, char, if.

Identifiers are user-defined names for variables, functions, etc. They are created by the programmer to identify different program elements. Examples: myAge, calculateSum.

7. What are constants in C, Give examples of different types of constants ?

A **constant** is a fixed value that cannot be changed during program execution.

- **Integer constants:** Whole numbers like 10, -50, 0.
 - **Floating-point constants:** Real numbers like 3.14, -2.5.
 - **Character constants:** Single characters enclosed in single quotes like 'A', '7', '\$'.
 - **String constants:** A sequence of characters enclosed in double quotes like "Hello", "C Program".
-

8. Explain the difference between implicit and explicit type conversion with examples ?

Implicit type conversion, also known as type promotion, is done automatically by the compiler. It converts a lower data type to a higher one to avoid data loss.

Example: converting an int to a double.

```
int num1 = 10;
double num2;
num2 = num1; // implicit conversion, num2 becomes 10.0
```

Explicit type conversion, or type casting, is done manually by the programmer using a cast operator. It forces the conversion of a value from one type to another.

Example: converting a double to an int.

```
double numA = 10.5;
int numB = (int)numA; // explicit conversion, numB becomes 10
```

9. What are escape sequences, Write the purpose of \n, \t, and \" ?

Escape sequences are special character combinations used in C to represent non-printable characters or to add special formatting to strings. They begin with a backslash \.

- `\n`: Represents a **newline character**, moving the cursor to the beginning of the next line.
 - `\t`: Represents a **horizontal tab**, inserting a tab space.
 - `\"`: Inserts a **double quote character** into a string.
-

10. What is the difference between `printf()` and `scanf()` functions in C, Give examples of each ?

- The **`printf()`** function is used to **display output** on the screen. It formats and prints data.
 - Example: `printf("Hello, World!");`
 - The **`scanf()`** function is used to **read input** from the user. It reads formatted data from the keyboard.
 - Example: `scanf("%d", &age);` (reads an integer value from the user and stores it in the age variable).
-

Chapter 1 Sample Programs

Program 1: Basic Input and Output with Variables

```
#include <stdio.h>

int main() {
    int num1, num2, sum;

    // Prompt the user for input
    printf("Enter the first number: ");
    scanf("%d", &num1);

    printf("Enter the second number: ");
    scanf("%d", &num2);

    // Calculate the sum
    sum = num1 + num2;

    // Display the result
    printf("The sum of %d and %d is: %d\n", num1, num2, sum);

    return 0;
}
```

Output:

```
Enter the first number: 10
Enter the second number: 25
The sum of 10 and 25 is: 35
```

Program 2: Demonstrating Different Data Types and sizeof

```
#include <stdio.h>

int main() {
    int age = 22;
    float pi = 3.14;
    char grade = 'A';
    double big_number = 12345.6789;

    printf("Age: %d\n", age);
    printf("Pi: %f\n", pi);
    printf("Grade: %c\n", grade);
    printf("A big number: %lf\n", big_number);

    printf("\nSize of int: %zu bytes\n", sizeof(int));
    printf("Size of float: %zu bytes\n", sizeof(float));
    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of double: %zu bytes\n", sizeof(double));

    return 0;
}
```

Output:

```
Age: 22
Pi: 3.140000
Grade: A
A big number: 12345.678900
Size of int: 4 bytes
Size of float: 4 bytes
Size of char: 1 bytes
Size of double: 8 bytes
```

Program 3: Explicit Type Conversion (Type Casting)

```
#include <stdio.h>

int main() {
    int total_items = 5;
    int students = 2;
    float avg_items;

    // Implicit conversion (incorrect result)
    avg_items = total_items / students;
    printf("Without typecasting (incorrect): %f\n", avg_items);

    // Explicit type conversion (correct result)
    avg_items = (float)total_items / students;
    printf("With typecasting (correct): %f\n", avg_items);

    return 0;
}
```

Output:

```
Without typecasting (incorrect): 2.000000
With typecasting (correct): 2.500000
```